

Introducción a la Ingeniería de Software

Ingeniería de Sistemas

Universidad Austral — 2026

Un poco de historia

Margaret Hamilton

El término "**Ingeniería de Software**" lo popularizó Margaret Hamilton en los años 60.

Dirigía la división de software del **MIT Instrumentation Laboratory**, que en 1961 tomó el contrato de la NASA para el sistema de vuelo del programa **Apollo**.

Usaba el término, en parte, para reclamar la misma legitimidad que tenían las otras ingenierías del proyecto.

El software de Apollo 11

Durante el descenso del módulo lunar, la computadora se saturó y empezó a tirar las alarmas **1201** y **1202**.

El sistema **descartó las tareas de menor prioridad** y siguió ejecutando las críticas. El alunizaje continuó.

Eso no fue suerte: fue una decisión de diseño tomada años antes.

La "crisis del software"

Las primeras conferencias de Ingeniería de Software fueron organizadas por la **OTAN**: Garmisch (1968) y Roma (1969).

El diagnóstico era el mismo en todas partes:

- El **hardware** mejoraba rápido, era barato y confiable 😎
- El **software** llegaba tarde, costaba de más y fallaba 😬

¿Se resolvió?

Casi 60 años después seguimos discutiendo lo mismo:

- Proyectos que se atrasan
- Sistemas que nadie se anima a tocar
- Estimaciones que no se parecen a la realidad

Lo que sí cambió: hoy tenemos **herramientas y prácticas** que hacen medible y manejable buena parte del problema.

¿Qué es la Ingeniería de Software?

Dos definiciones

Software engineering is the application of an empirical, scientific approach to finding efficient, economic solutions to practical problems in software.

— **Dave Farley**

...discipline that is concerned with all aspects of software production, from the early stages of system specification to maintaining the system after it has gone into use.

— **Ian Sommerville**

Lo que tienen en común

- **Empírica** — se decide con evidencia, no con opinión
- **Económica** — hay restricciones de tiempo, plata y gente
- **Completa** — incluye el mantenimiento, no termina en el *release*

Ingeniería ≠ Programación

Sommerville distingue el software **profesional** por dos rasgos:

- Está **hecho para que lo use otro**, no su autor
- Lo desarrollan **equipos**, no individuos

De ahí se desprende todo lo demás: especificación, diseño, documentación, convenciones, procesos de cambio.

El costo está en el mantenimiento

Escribir la primera versión es la parte barata.

Distintos estudios ubican el mantenimiento entre el **60% y el 80%** del costo total de un sistema a lo largo de su vida.

*Optimizamos para **leer y cambiar**, no para escribir.*

El proceso de desarrollo

Definición

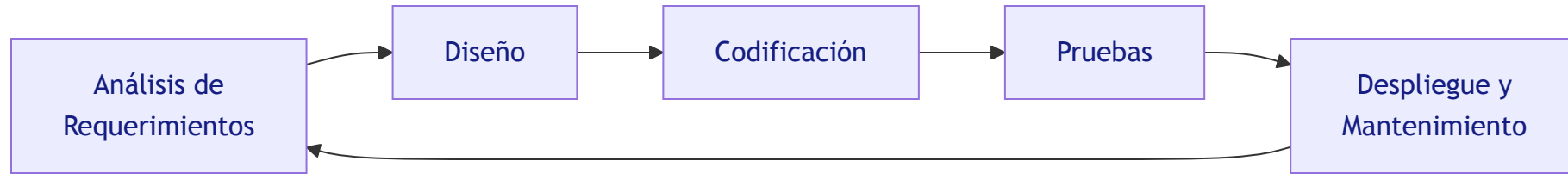
*El conjunto de **tareas**, la **secuencia** y el **flujo** de esas tareas, sus **entradas y salidas**, y las **condiciones previas y posteriores** de cada una de las tareas involucradas en producir un software.*

Estandarizado en **ISO/IEC/IEEE 12207** — *Software life cycle processes*.

Áreas que cubre

- Diseño
- Código y tests de unidad
- Testeo funcional y de integración
- Documentación
- Lanzamiento (*release*)
- Soporte y mantenimiento
- Administración de cambios

El ciclo de vida



Ninguna fase es un momento

En un equipo real, en la misma semana hay gente:

- analizando el requerimiento de la iteración que viene
- diseñando el módulo de la iteración actual
- arreglando un *bug* de lo que ya está en producción

El ciclo de vida describe **tipos de actividad**, no un calendario.

Personas

El problema de la comunicación

Con n personas hay $n \times (n-1) / 2$ canales de comunicación posibles:

Personas	Canales
3	3
5	10
10	45
20	190

La Ley de Brooks

Adding manpower to a late software project makes it later.

— Fred Brooks, *The Mythical Man-Month* (1975)

Sumar gente agrega canales de comunicación y tiempo de *onboarding* antes de agregar capacidad de producción.

Proceso, Producto y Personas

Proceso

Estandarizar para ser **previsibles** y evitar errores. No hay uno solo que sirva para todos los casos.

Producto

Va más allá del código: diseños, documentación, tests, instalación, infraestructura.

Personas

Equipos con capacidades diversas. Coordinar el esfuerzo **es** parte del trabajo de ingeniería.

Lo que es genuinamente difícil

Esencia y accidente

Fred Brooks, en **No Silver Bullet** (1986), separa dos tipos de dificultad:

- **Accidental** — la que ponen nuestras herramientas
(compilar lento, configurar el ambiente, deployar a mano)
- **Esencial** — la del problema en sí
(entender qué hay que construir, y que sea correcto)

Las herramientas atacan la accidental. Es la esencial la que domina.

Programar es construir una teoría

Peter Naur (1985): lo valioso de un programa no es el texto del código, es la **teoría** que el equipo tiene en la cabeza sobre por qué está hecho así.

- El código es un artefacto **parcial** de esa teoría
- Cuando el equipo se va, la teoría se pierde
- Un programa sin su teoría es muy difícil de modificar bien

¿Y con la IA?

Vuelta a Brooks

La IA ataca, sobre todo, la complejidad **accidental**:

- escribir *boilerplate*, traducir entre APIs, recordar sintaxis
- generar tests, andamiaje, migraciones mecánicas

Lo **esencial** sigue igual de difícil:

- decidir **qué** hay que construir
- decidir si lo construido es **correcto**
- sostener la coherencia del sistema en el tiempo

Lo que dice la evidencia

DORA 2025 — 90% de adopción. El *throughput* subió, pero la **inestabilidad de las entregas también**.

METR 2025 — en un experimento controlado, devs experimentados fueron **19% más lentos** con IA en repos que conocían bien... y estimaron después que habían sido **20% más rápidos**.

*La IA es un **amplificador**: mejora a los equipos que ya estaban bien y agrava los problemas de los que no.*

Naur en la era de la IA

Si lo valioso es la **teoría** y no el texto del código...

*...entonces código que nadie del equipo entiende es un programa **sin su teoría**.*

Generar código es más barato que nunca. **Construir la teoría compartida, no.**

Por eso los *code reviews*, los tests, la trazabilidad y la documentación valen **más** que antes, no menos.

En esta materia

Pueden usar IA. No la vamos a prohibir.

Pero en la práctica hay **revisiones constantes, grupo por grupo:**

- El código es **de ustedes** y lo defienden en la revisión
- Si no pueden explicar **por qué está así**, no lo entreguen
- Evaluamos **decisiones de diseño**, no cantidad de líneas

Lectura para la clase 2

"Programming as Theory Building" — Peter Naur, 1985 (14 páginas)

👉 <https://ratfactor.com/papers/naur1>

Y una reflexión para contrastar:

👉 <https://emptysqua.re/blog/programming-as-theory-building/>

Lo que viene

Unidad	Tema
2	Buenas prácticas: trazabilidad, SCM, tests, automatización, ambientes
3	Requerimientos y estimaciones
4	Arquitectura de software
5	Metodologías de desarrollo

Y de fondo, siempre: **el TP.**

¿Preguntas?